

Lab 3: Linear and Circular Queues

Module	ET4047
Date	27/09/2026
Lab No.	Lab 3
Author	Luke Griffin

1 Lab tasks

Work in your **Linear_Queue_Example** project unless a task says otherwise. Save a copy of the sketch after each task (for example **Task3.ino**, **Task4.ino**) so you can go back to it. Paste the Serial Monitor output into your lab report for every task marked **(record)**.

Task 1: Run and compare both examples

1. Run both example sketches in Wokwi. **(record)** Paste both outputs into your report.
2. The two outputs are identical. Explain in one or two sentences why this test cannot reveal any difference between the two designs.

Your answer:

Task 2: Add a `printQueue()` function

Debugging a queue is much easier when you can see inside it. Add a function to the linear sketch with this signature, placed **above** `setup()`:

```
void printQueue(Queue *q);
```

It should print **front**, **rear** and **count**, then every slot of the array. Show the value for slots that are **in** the queue and `[--]` for slots that are not. For example:

Required format

```
front=2 rear=4 count=3 | [--] [--] [30] [40] [50]
```

Hint

In the linear queue, slot **i** is part of the queue when **count > 0 and front <= i <= rear**. Use a **for** loop over all **MAX_QUEUE_SIZE** slots.

Task 3: Expose the problem

1. Change `#define MAX_QUEUE_SIZE 10` to `5`.
2. Replace the body of `setup()` (keep `Serial.begin(9600);`) with the test sequence below.
3. Run it. **(record)** Paste the output into your report.

Task 3 test sequence

```
initializeQueue(&myQueue);

enqueue(&myQueue, 10);
enqueue(&myQueue, 20);
enqueue(&myQueue, 30);
enqueue(&myQueue, 40);
enqueue(&myQueue, 50);
printQueue(&myQueue);
Serial.println();

dequeue(&myQueue);
dequeue(&myQueue);
printQueue(&myQueue);
Serial.println();

enqueue(&myQueue, 60);
enqueue(&myQueue, 70);
printQueue(&myQueue);
Serial.println();
```

Answer: (a) How many elements are in the queue when 60 is rejected? (b) Which line of code causes the rejection? (c) Would the problem go away if you made `MAX_QUEUE_SIZE` bigger? Explain.

Your answer:

Task 4: Fix 1, shift elements on dequeue

Modify `dequeue()` so that after the front element is saved, every remaining element slides **one place to the left**. `front` then always stays at 0, and the freed slot moves to the end of the array where `enqueue()` can reuse it.

- Write a `for` loop that copies `items[i + 1]` into `items[i]` for every element in the queue. Should it run forwards or backwards this time? Why?
- Remove the `q->front++` line and decrement `rear` instead.
- Change `isFull()` to `return q->count == MAX_QUEUE_SIZE;`. This is the clearest definition of “full”.

Re-run the Task 3 test. **(record)** 60 and 70 should now be accepted. Your final `printQueue()` line should show:

Expected final line

```
front=0 rear=4 count=5 | [30] [40] [50] [60] [70]
```

Think about it

This works, but what does it cost? If the queue holds n elements, how many copy operations does one `dequeue()` now perform? Record your answer in your lab report.

Task 5: Fix 2, convert the linear queue to a circular queue

Start from your **Task 3** version (not Task 4) and turn it into a circular queue yourself.

Do not copy the circular example; use it only to check your work afterwards.

1. `isFull()`: use `count` instead of `rear`.
2. `enqueue()`: replace `rear++` with a wrap-around increment using `%`.
3. `dequeue()`: replace `front++` with a wrap-around increment. Is the “reset when empty” block still needed? Try it with and without.
4. `enqueueToOppositeEnd()`: delete the shifting loop entirely. Step `front` backwards with wrap-around and write the value there. If the queue was empty beforehand, make `rear` equal `front` too.
5. `printQueue()`: your Task 2 test `front <= i <= rear` no longer works once `rear` wraps behind `front`. Fix it using the hint below.

Hint for `printQueue()`

Work out how far slot `i` is from the front, moving forwards with wrap-around:

```
int offset = (i - q->front + MAX_QUEUE_SIZE) % MAX_QUEUE_SIZE;
```

Slot `i` is part of the queue when `offset < q->count`.

Re-run the Task 3 test. **(record)** The final line should show 60 and 70 stored in slots 0 and 1:

Expected final line

```
front=2 rear=1 count=5 | [60] [70] [30] [40] [50]
```

Task 6: Stress test

A single test is not proof. Add the function below to your sketch and call it at the end of `setup()` (after `destroyQueue(&myQueue);`). It keeps one element in the queue at all times, so the “reset when empty” trick never rescues it, and then pushes 20 more values through a 5-slot queue, checking they come out in the right order.

Task 6 stress test

```
void stressTest(Queue *q)
{
    int next = 1;        // next value to enqueue
    int expected = 1;    // value we expect dequeue() to return
    int failures = 0;

    // Keep one item in the queue so it never becomes empty
    enqueue(q, next++);

    for (int round = 1; round <= 10; round++)
    {
        for (int i = 0; i < 2; i++)
        {
            if (!enqueue(q, next++)) failures++;
        }
        for (int i = 0; i < 2; i++)
        {
            if (dequeue(q) != expected++) failures++;
        }
    }

    Serial.print("Stress test finished. Failures: ");
    Serial.println(failures);
}
```

1. Run the stress test on the **original** linear queue (Task 3 version). **(record)** Record the number of failures.
2. Run it on your **Task 4** and **Task 5** versions. **(record)** Both must report **Failures: 0**.
3. Look at the original version's output. At which value does the first "Queue is full" error appear, and why at that point?

Your answer:

Task 7 (extension): Interactive queue over Serial

Make the queue interactive using the Wokwi Serial Monitor input box. In `loop()`, read a line with `Serial.readStringUntil('\n')` and respond to these commands, calling `printQueue()` after each one:

Command	Action
e 42	enqueue 42
f 7	enqueueToOppositeEnd 7
d	dequeue and print the value
p	peek and print the value
c	clear (destroyQueue)

Use it to demonstrate the wrap-around live: fill the queue, dequeue a few, then enqueue again. Hint: `String.toInt()` converts text to a number, and `Serial.available()` tells you when input has arrived.